Assessment Brief

CA1 – Database Design Project

Module Name	Lecturer Name
Advanced Databases	Michael McAndrew

Title of Brief
CA1 – Database Design Project

Percentage of Overall Grade	65%
Date handed out	9th November 2026
Code due date	6th December 2026
Report due date	13th December 2026
Individual or Group	Individual

Problem to Solve

You are hired as an early engineer at a growing software company, and **your task is to build the database system behind their app with MySQL.** People will use the app on Android, iOS or the web, and the app reads from and writes to the database every time they use it.

You will choose **one** of the following apps to build the database for:

- **Goodreads:** A social network for readers to track, rate and review books
- **Steam:** A store for PC games, where players buy, play and review games

You will be provided with a **starter database** containing real data for each app, and your task is **to extend this database** to support the features that the app needs.

Read your app's README carefully before you start. Many features can be built in more than one way, and a large part of this project is deciding on the right database design for the app and being able to justify it.

Report

Throughout the project, you will write a report, `report.pdf`, where you explain and justify the decisions you make. Each part of the specification below describes what your report should cover.

Specification

Part 1: Database Design [20%]

Design a database that supports all of the features in your app's README. Draw an **Entity-Relationship Diagram (ERD)** of your database system (including the starter tables that your

design uses) and write the table creation queries to the `schema.sql` file provided in the template.

Be sure to define schema for tables **precisely**, to make the database robust against data integrity issues and well-normalised. In your report, explain the main decisions behind your design, particularly where a feature could be modelled in more than one way, or where you have chosen not to fully normalise.

Part 2: Querying the Database [25%]

The application code will query the database that you built to read, insert, update, and delete data so that people can use the app quickly. Write the queries listed in your app's README, and group them by feature in `queries.sql`.

Implement these features using **an appropriate mix of queries, stored procedures, triggers, and views**. Where appropriate, enforce the rules in your app in the database itself to maintain data integrity. Include a brief comment above each explaining your choice.

Part 3: Query Optimisation [10%]

It is important as the database grows that queries made by the client applications remain fast. Think about how people are likely to use your app, and identify the queries that will run most often, and the queries that are slow or may become slow as the database grows. With the help of the `EXPLAIN` query, analyse these queries and add the appropriate indexing strategies to ensure that they remain fast.

In your report, show the `EXPLAIN` output before and after each optimisation, and explain why you chose each index. The tables that you create will start empty, so your `data.sql` file must seed them with enough data to show the queries working.

Part 4: Transactions and Concurrency [5%]

As your app grows, many people will use the database at the same time, and the same person may use it from more than one device at once. Analyse the queries you implemented in Part 2 and identify which operations require transaction management and concurrency control to keep the data consistent. In your report, justify the isolation level you chose in each case.

Part 5: Security and Privacy [15%]

Keeping user data secure is of the utmost importance. Implement the following security measures to help keep users' data secure:

User Roles and Permissions

Different people at the company need different levels of access to the database. For example, some staff only need to read data, while others need to edit some data but not all

of it. Your app's README lists the groups of people who need access. Create a database role for each group, and give each role only the access that it needs. For each role, also create an example user and grant the role to them. Include the SQL commands to create these roles and users in your `schema.sql` file, and explain in your report what each role can and cannot do.

SQL Injection

SQL injection is one of the most common and dangerous web application vulnerabilities. Protect every query that takes user input by using parameterised stored procedures or prepared statements.

Data Deletion

In the EU, when a user deletes their account with a service, you are required by law to delete all personally identifiable data relating to that user. Write a query to delete a given user, and ensure that when the user is deleted, all of their personal data is also deleted from the database. Some of a user's data may be connected to other users' data, so explain in your report how you handled this.

Part 6: Extra Mile [10%]

This is an opportunity for you to design and implement a feature that your app currently doesn't have but that you wish it did. Describe the feature in your report and include it in your ERD. Add the schema changes, queries and sample data that it needs to your `schema.sql`, `queries.sql` and `data.sql` files.

Presentation and Documentation [15%]

You will give a demonstration in class in the week of 7th December covering your database design, executing the queries that you wrote, query optimisation techniques, transaction and concurrency considerations, and security and privacy best practices.

During your demonstration, you will be asked to explain how your work in each part of the project works. Make sure that you understand all of the code you submit. **If you can't explain your work for a part, your grade for that part will be capped at 40%.**

You must attend the demonstration in class to receive a grade for your submission.

You will also record a video demo of up to 5 minutes that shows every feature of your database working. You are not required to explain how each feature works in detail for the video, just a short explanation of each feature and evidence of it running.

Submission

By **Sunday 6th December at 23:59**, submit your code and ERD:

- `erd.png`/`erd.pdf`: your entity-relationship diagram. This can be drawn using a tool like draw.io, dbdiagram.io or even hand drawn if you'd prefer, but entities, relationships and their types must be clearly shown.
- A zip file of your repository, which should include:
  - `schema.sql`: a list of the create table, create index, create role and create user commands
  - `queries.sql`: a list of all of the queries, stored procedures, triggers, and views, grouped by feature
  - `data.sql`: any data queries required to seed the database with sample data

By **Sunday 13th December at 23:59**, after the demonstrations, submit your report and video:

- `report.pdf`: a rigorous report. Explain the decisions you made in each part of the project, as described above. Also write about the evolution of your project as you worked on it, the scope and limitations of your work, and potential future work. Write for a technical audience who has already taken this course. Aim for around 1,500 words. State the app you chose at the top of the report.
- The video demo, up to 5 minutes long, showing every feature working.

For each day that either submission is late, 5% will be deducted. Assignments submitted more than 7 days late will not be accepted.

Academic integrity guidance for this project is available on the module website.

Marking Scheme

Component	Weight	What is assessed
Part 1: Database Design	20%	The ERD shows every entity and relationship the app needs, with the correct cardinality. The schema can store data for all features for the app, and suitable data types are chosen throughout. Appropriate constraints are used to prevent invalid data from being stored and to maintain data integrity. The schema is well-normalised, and any exceptions are justified.
Part 2: Querying the Database	25%	All of the queries listed in the chosen app's README are well implemented. Each feature uses a suitable query, stored procedure, trigger or view, and the choice is justified. Queries are readable and grouped by feature.
Part 3: Query Optimisation	10%	Queries that will slow down as the database grows are identified, based on how the app is used. EXPLAIN output is used to show the effect of each optimisation. Indexes are added and are suitable for the queries they support, and each one is justified.
Part 4: Transactions and Concurrency	5%	Operations that need transactions are correctly identified. Isolation levels are appropriate and justified.
Part 5: Security and Privacy	15%	Each role has the access it needs and follows the principle of least privilege. All queries that take user input are protected against SQL injection. Deleting a user removes all of their personal data, and the handling of connected data is justified.
Part 6: Extra Mile	10%	The feature is clearly described and included in the ERD and schema. The feature's schema changes and queries work correctly. The feature is original, useful, and more than a small change to an existing feature.
Presentation and Documentation	15%	The demonstration clearly explains the design and the choices made in each part. The student can explain how their queries work when asked. The video shows every feature working. The report explains and justifies the decisions made, and reflects on the evolution, scope, limitations and future of the project.

Grades for Parts 1 to 6 are capped at 40% for any part whose work you can't explain in the demonstration.